Nirma University
Institute of Technology



Course Code :- 6CS102CC26
Course Name :- Data Structure

Innovative Assignment 1

Student Name
Vinay Thakor

Student Roll No.
26MCA061

Faculty In Charge
Prof. Nirav Patel

BLOCKSTRUCT

An Interactive Tool for Exploring Blocks, Merkle Trees, Tamper Detection and Network Propagation

Innovative Assignment Report

1. Introduction

A blockchain is an append-only list of blocks in which every block is storing the hash of the block before it, to a significant degree. Seeing that a hash changes completely when even one character of its input changes, editing an old block breaks the link to every block that's following, and the damage can, to some extent, be found by recomputing hashes, in many respects; and thus, each block also summarises its transactions with a single Merkle root, a hash built from a binary tree of transaction hashes, so one short value commits to the whole list. A block is accepted only after a proof-of-work step, in which the miner searches for a nonce that's giving the block header a hash with a needed number of guiding zeros.

The project is implementing these ideas as an interactive visualisation called BlockStruct, written in plain HTML, CSS (Tailwind) and JavaScript with SVG graphics and no other libraries, broadly speaking. The SHA-256 function is used by hand and runs in the browser, so nothing is sent to a server, on the whole. The page is starting with a chain of four mined blocks and lets the user mine new blocks, queue transactions in a pool, search the chain, confirm a Merkle proof for any transaction, tamper with a block and check the chain, and broadcast a block across a small peer-to-peer network of nodes.

Six test cases were run, from a one-transaction genesis block to the broadcast of a block over the node graph, in actual application; indeed, they're reported in Section 6. Every hash and counter employed in them can, in practice, be reproduced in the tool, so each finding can be validated by hand, in real-world scenarios.

2. Input Representation and Data Structures

The data are being listed by table 1 that the program is working with, in real-world scenarios; in fact, let m be the number of transactions in a block and N the number of blocks in the chain, on the whole.

Table 1, Notation employed by BlockStruct

Name	Type / Size	Meaning
tx	string	One transaction in the form "sender → receiver : amount", typed by the user or created at random.
pool	array of string	Transactions waiting to be mined; mining the pool moves all of them into one new block, in actual application.
block	object	Holds the index i, timestamp ts, previous hash prev, Merkle root root, nonce, the list txs and its own hash, to a significant degree.
chain	array, N blocks	The ordered list of blocks; chain[0] is the genesis block whose prev is normally 64 zeros, in actual application.

Name	Type / Size	Meaning
merkle(txs)	array of levels	The m transaction hashes are held by level 0, which is mainly why each higher level is holding the parent hashes, up to a single root, in real-world scenarios.
bad	object	Maps a block index to the list of validation errors found for that block, in real-world scenarios.
nodes, edges	arrays	The peer-to-peer graph, like each node stores the height h of the chain it has received; two peers are being joined by each edge, in real-world scenarios.

The function sha(m) computes SHA-256 from first axioms, such as it pads the message, expands each 64-byte chunk into 64 words and runs 64 rounds of compression, employing constants obtained from the square and cube roots of the first primes, on the whole; and thus, the function merkle(txs) hashes every transaction, then repeatedly hashes pairs of neighbours; when a level has an odd number of hashes, the last hash is paired with itself, in many respects. The functions renderStrip(), renderDetail() and renderNet() redraw the chain strip, the block details and the network after every change, as recapped in Section 4; indeed, the block hash is the SHA-256 of the string i|ts|prev|root|nonce, and mine(txs, prev, i) augments the nonce until that hash begins with 00.

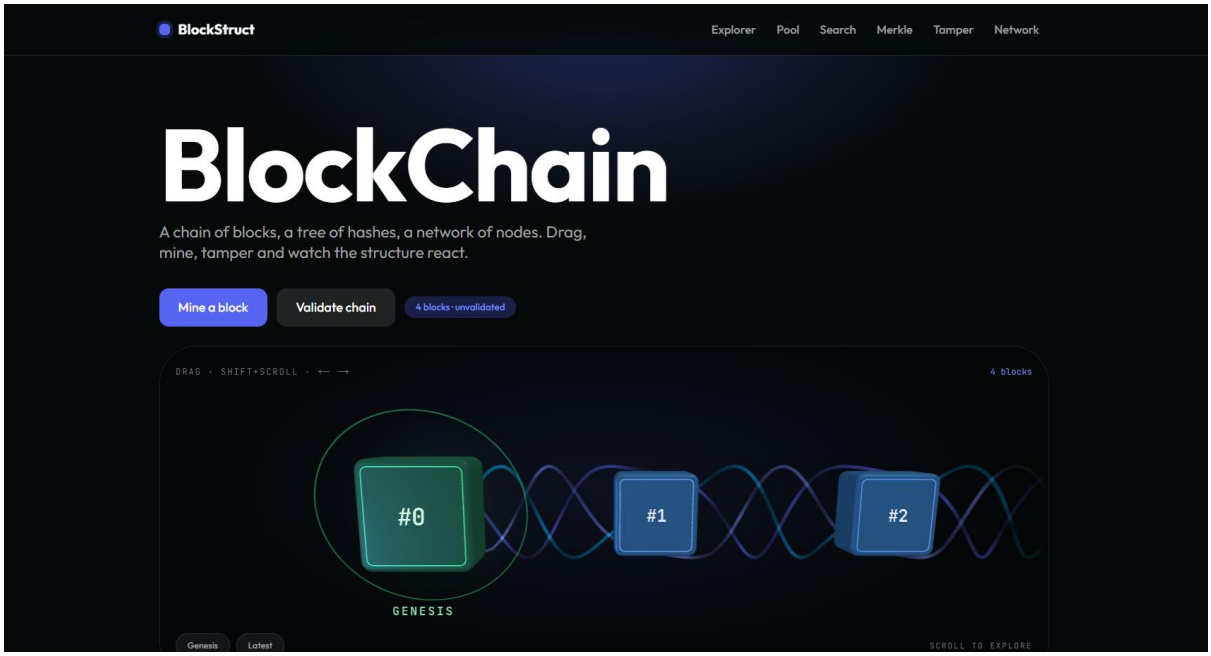


Figure 1, Main window of the BlockStruct tool showing the title likely, the action buttons and the strip of linked blocks

3. Block, Merkle Tree and Mining Algorithms

The following methods are being employed by the tool; in fact, each is deterministic for a given input, so the same transactions and header always samely hash, on the whole.

- **Hash the transactions.** Every transaction string is usually hashed with SHA-256 to form the leaves of the Merkle tree, on the whole; a block with no transactions is, for all practical purposes, employing the hash of the empty string.

- **Build the Merkle tree.** Stop when one hash remains; this is the Merkle root; indeed, if a level has an odd number of hashes, the last hash is paired with itself, to a significant degree. Pair neighbouring hashes left to right, concatenate each pair and hash the finding to form the next level, in actual application.
- **Mine the block.** Set the nonce to 0, hash the header i|ts|prev|root|nonce and increase the nonce until the hash starts with two hexadecimal zeros (8 zero bits), broadly speaking; and thus, the nonce and the winning hash are, in effect, being stored by the block.
- **Prove the chain.** Each in turn block check three things, namely the Merkle root recomputed from the transactions equals the stored root; the header hash recomputed equals the stored hash; and prev equals the hash of the coming before block, on the whole.
- **Confirm a Merkle proof.** Hash the chosen transaction, then combine it with its sibling hash at each level (left or right according to the parity of its index) until the top is reached; the proof is valid if the rebuilt value equals the block root, in many respects.
- **Pass on a block.** The chosen node receives the block at once; in each round every node that already has it sends it over its edges to peers that don't, so the block is spreading in breadth-first order, in many respects.

A Merkle tree over m transactions is needing 2m − 1 hashes when m is a power of two, on the whole; indeed, as, well, a proof is needing only ⌈log2 m⌉ sibling hashes. Mining is a random search, namely every try succeeds with probability 16^(−d) for d directing hex zeros, broadly speaking; and thus, checking the chain costs O(T) hashes for T transactions in total, because each block recomputes its Merkle root and header. But still, with d = 2 the anticipated number of tries per block is 256; as, well, the cost grows by a factor of 16 for each extra zero, which is why even so, searching the chain takes O(T + N) time. Appending a block with a changed transaction is the only way to keep the chain valid, because the changed Merkle root is altering the header hash and therefore the prev link of every later block.

4. Visualising and Explaining the Result

The tool does not only print hashes. Each change is currently reflected on screen at once; so, every arrangement can, in practice, maybe be inspected, in many respects.

- **Block strip.** All i mean is probably, the chain is often drawn as a horizontal row of cards joined by connectors with a moving dot, to a significant degree. Each card is showing the block number, its hash and previous hash with the guiding zeros showed in green, the number of transactions and the nonce. The strip can be dragged, a new block animates into the place. And an invalid block is turning red with an Invalid tag.
- **Explorer.** Clicking a block opens its header, timestamp, nonce, hash, previous hash, Merkle root and the full transaction list with each transaction hash, on the whole.
- **Transaction pool.** The user types or randomly is creating transactions, removes any of them with a cross, and mines the whole pool into one block, broadly speaking.
- **Search.** A single box is matching a block number, a hash prefix or part of a transaction, and clicking a finding jumps to that block, to a significant degree.

- **Merkle proof view.** A message reports how many sibling hashes restore the root. The tree is generally drawn level by level; the path from the transaction to the root is, of course, green and the sibling hashes that complete the proof are mainly blue.
- **Tamper detection.** A block and transaction are chosen and rewritten, then the chain is checked; and thus, each block is getting a tick or a cross with the reasons, and the buttons Re-mine this block and Re-mine from here show how much work an attacker must redo, to a significant degree.
- **Network view.** Nodes sit on a circle joined by edges; a green packet travels along an edge when a block is typically passed on, and a node turns green when it holds the latest block, in real-world scenarios.

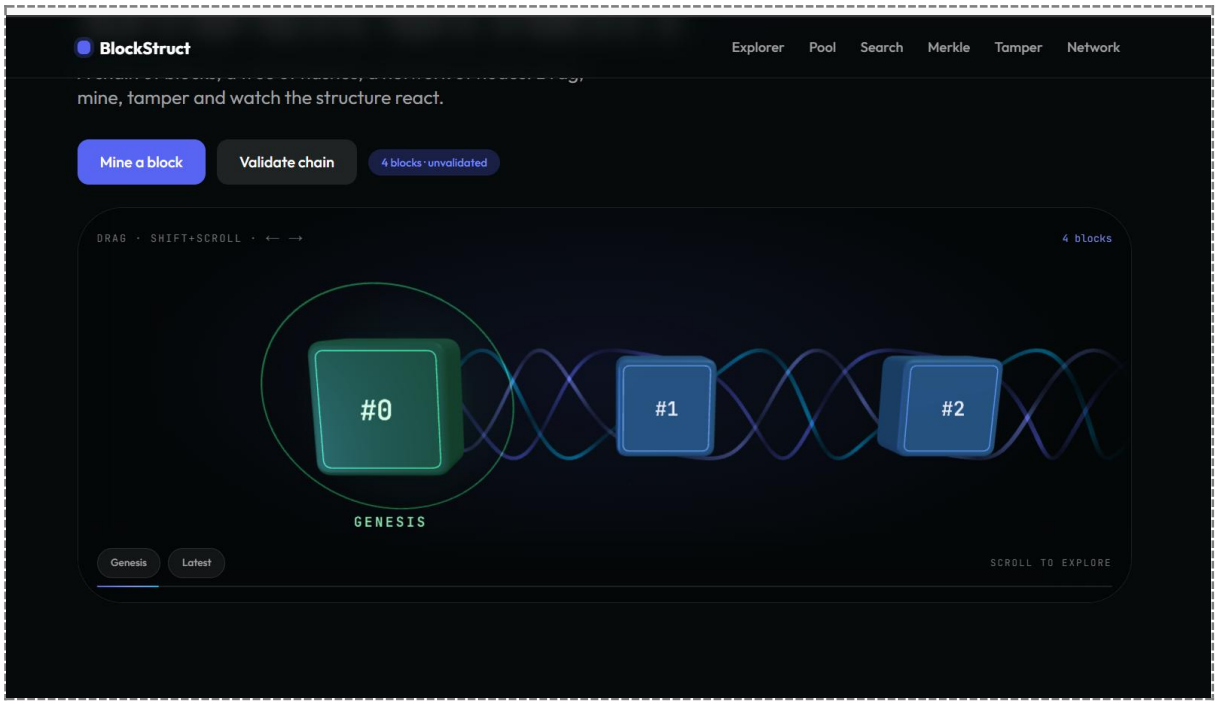


Figure 2, Explorer section with the details of a chosen block

5. Extra Features

5.1 Merkle Proof Visualisation

The Merkle section lets the user choose a block, click one of its transactions (or type any text) and press Verify, in many respects. If the text isn't in the block, the tool reports that no proof exists. Otherwise the function verify() walks from the leaf to the root. At every level it picks the sibling (index XOR 1, or the node itself when there's no right neighbour), combines the two hashes in the correct order and moves to the parent index. The nodes on the path are shown in green and the siblings in blue; in turn, also, the final message is stating whether the rebuilt root matches the block root. A block of eight transactions is needing only three sibling hashes instead of seven other transactions, which is the principal benefit of a Merkle tree.

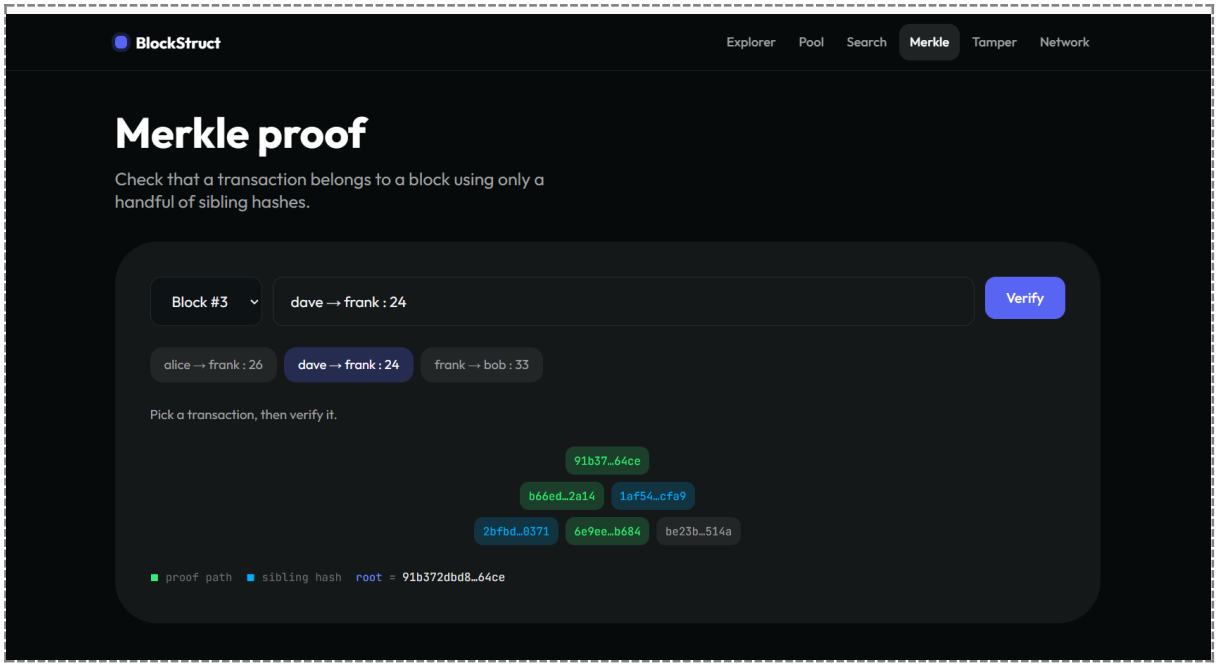


Figure 3, Merkle proof for one transaction, namely path in green, siblings in blue

5.2 Tamper Detection and Re-mining

The tool doesn't check automatically; the user must press Validate chain, so that the effect of the edit can be projected first, after a transaction is edited. Re-mining only the edited block is giving it a new valid hash but leaves the next block pointing at the old hash, so the break moves one block forward. Measured findings are given by section 6.4. My point is, re-mining from the edited block to the end repairs every link, at the cost of mining every later block again, to a significant degree. The two re-mine buttons recompute the proof of work.

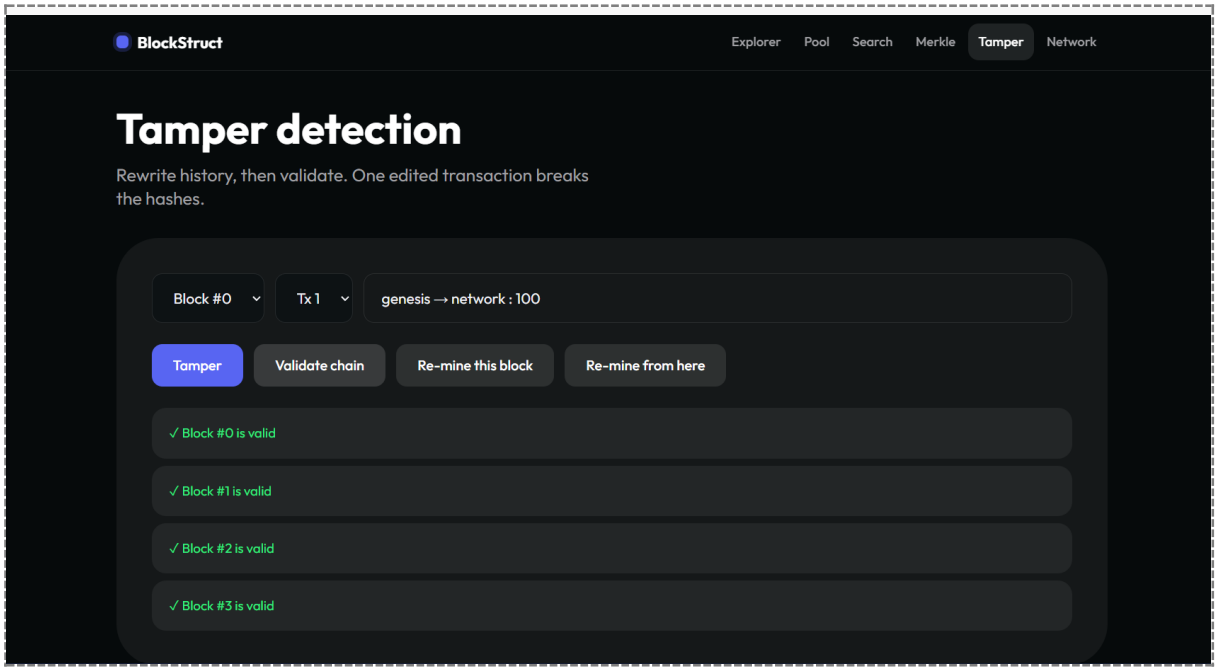


Figure 4, Chain validation after a transaction was edited: the altered block is normally marked invalid

5.3 Network Simulation

The network section is showing six nodes N1 to N6 joined in a ring with two extra links (N1, N4 and N2, N5), broadly speaking, and add node connects a new node to two existing ones and brings it up to date at once, up to a limit of twelve nodes, in real-world scenarios. Each round takes 700 ms and each packet 560 ms to travel, so the spread can (to some degree) be followed on screen; indeed, the user picks a node and presses Broadcast from node, which mines a block and sends it from that node. And a counter reports how many nodes hold the latest block.

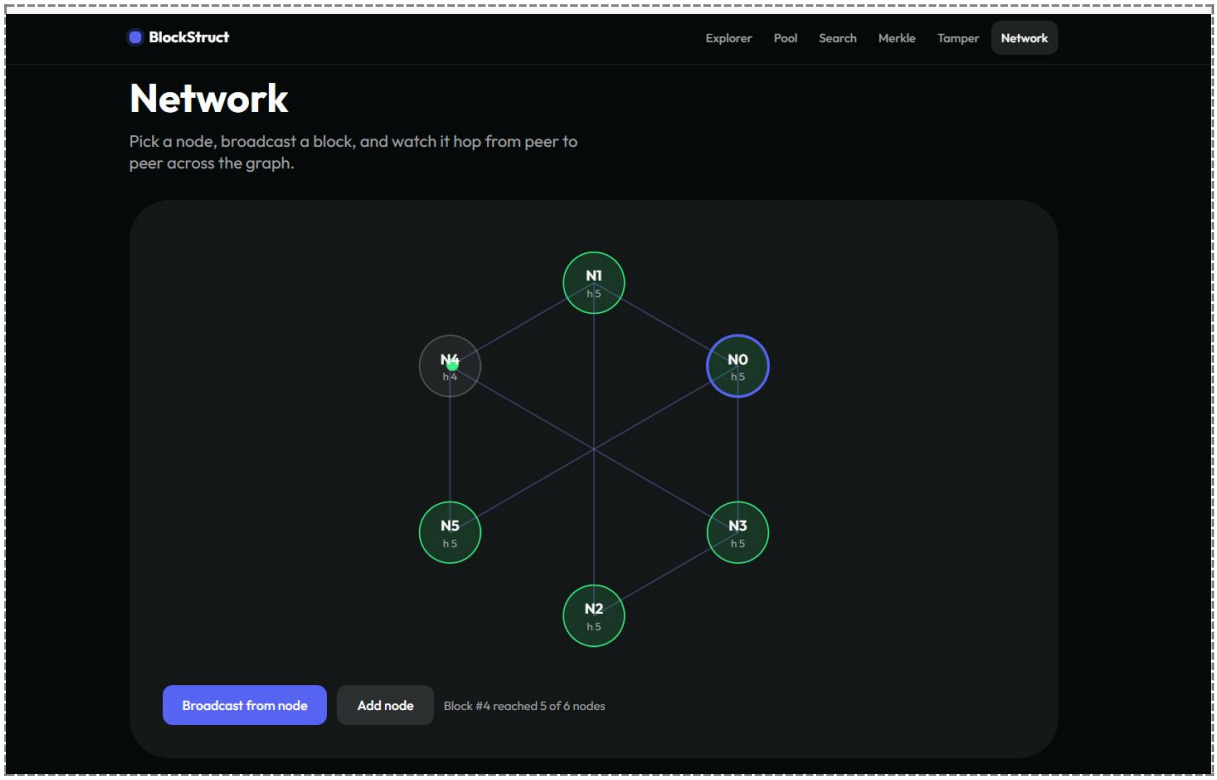


Figure 5, Network view while a block is often passed from node to node

5.4 Navigation and Usability

A sticky bar links to the Explorer, Pool, Search, Merkle, and Tamper sections and shows the section that's currently on screen, in real-world scenarios; indeed, the layout adapts to phones and tablets, the block strip scrolls by touch or mouse drag, on the whole. What's more, users who prefer lowered motion are getting the animations switched off.

6. Test Cases and Results

Six test cases were run. The SHA-256 implementation was verified against a reference implementation (Node.js crypto) for a few inputs, including the empty string and "abc", and gave same output, in many respects; and thus, the first three are explored closely in detail in Sections 6.1 to 6.3; the tamper detection, mining and network cases are covered in Sections 6.4 to 6.6, on the whole. In the tests the block timestamps were fixed (1 700 000 000 000 ms plus 1 000 ms per block index) so that every hash can be reproduced; in the live tool the timestamp is the current time, so nonces and hashes differ from run to run, on the whole.

6.1 Test Case 1 - Genesis Block with One Transaction

The genesis block is holding the single transaction "genesis → network : 100" and employs 64 zeros as its previous hash. One level is had by the Merkle tree with only one leaf, and the root equals the hash of the transaction itself. The proof therefore is needing 0 sibling hashes and is usually valid.

Table 2 - Result of Test Case 1

Item	Value
Transactions (m)	1
Merkle levels / hashes	1 level, 1 hash
Merkle root	c99ac830…507b0e (equal to the hash of the transaction)
Sibling hashes in proof	0 (proof valid)
Nonce found	68 (69 attempts)
Block hash	00dbb08f…7d1f3b

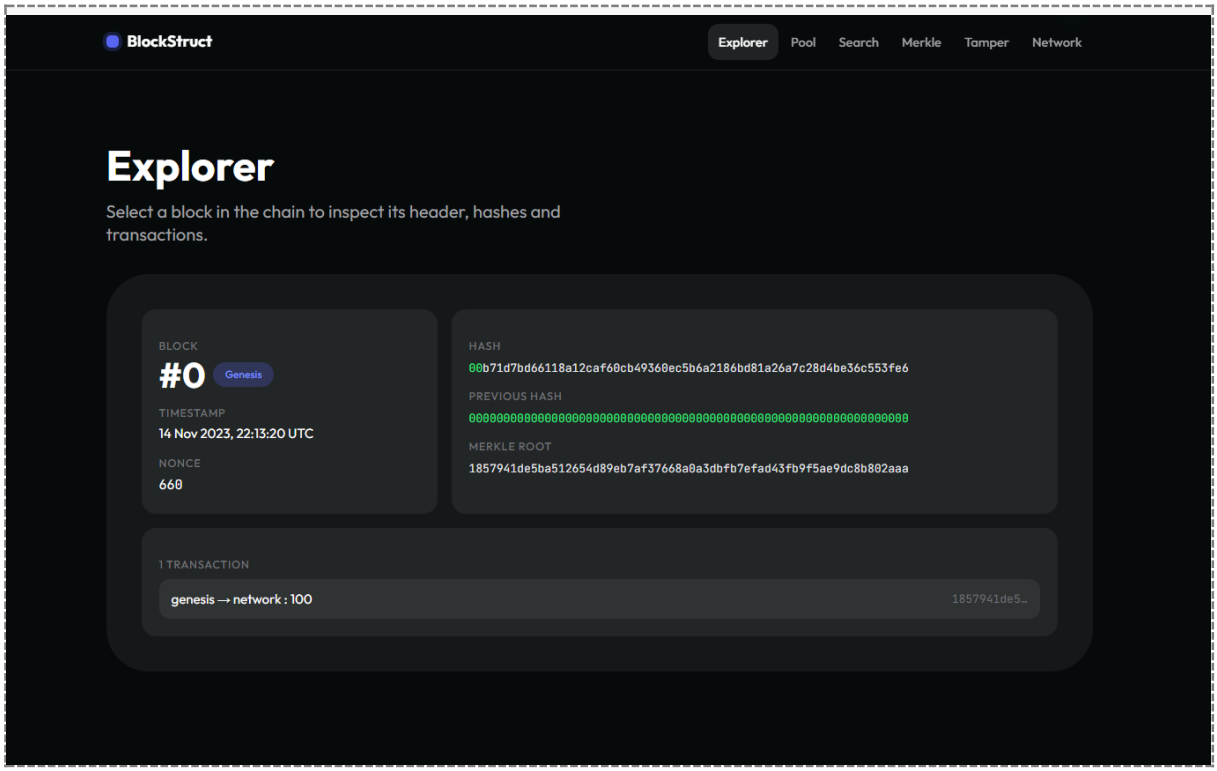


Figure 6 - Test Case 1, including genesis block with one transaction and its Merkle root

6.2 Test Case 2 - Block with Three Transactions

Block #1 is holding alice → bob : 12, carol → dave , like 7 and erin → frank : 30, in many respects; the three leaf hashes are generally paired as (T1, T2) and (T3, T3), because the third hash has no partner, to a significant degree. On level 1 and one root on level 2, this gives two parents; so, that said, the tree has 6 hashes in 3 levels; a valid proof of 2 sibling hashes is , to be sure, had by every transaction. Table 3 is showing the first 8 hex digits of every hash in the tree and Table 4 the proof for the third transaction.

Table 3 - Merkle tree of Test Case 2 (first 8 hex digits)

Level	Hashes	How they're formed
0 (leaves)	42b22964, ccfea670, b45dbfaf	SHA-256 of each transaction
1	68f05b84, 4712f745	hash(T1+T2), hash(T3+T3)
2 (root)	bcff71a4	hash(level-1 left + right)

Table 4 - Merkle proof for erin → frank , including 30

Level	Own index	Sibling used	Combined as
0	2	index 2 (itself)	hash(T3 + T3) → 4712f745
1	1	index 0 (68f05b84)	hash(68f05b84 + 4712f745) → bcff71a4
2	0	-	rebuilt root bcff71a4 equals the block root

Result, like the block was mined with nonce 41 (42 tries) and its hash is currently 0030be7e…a9423, in real-world scenarios; indeed, the proof of every transaction is valid with 2 sibling hashes. The duplication of the last leaf is the reason why the proof of the third transaction is employing the hash of that transaction itself as its first sibling, on the whole.

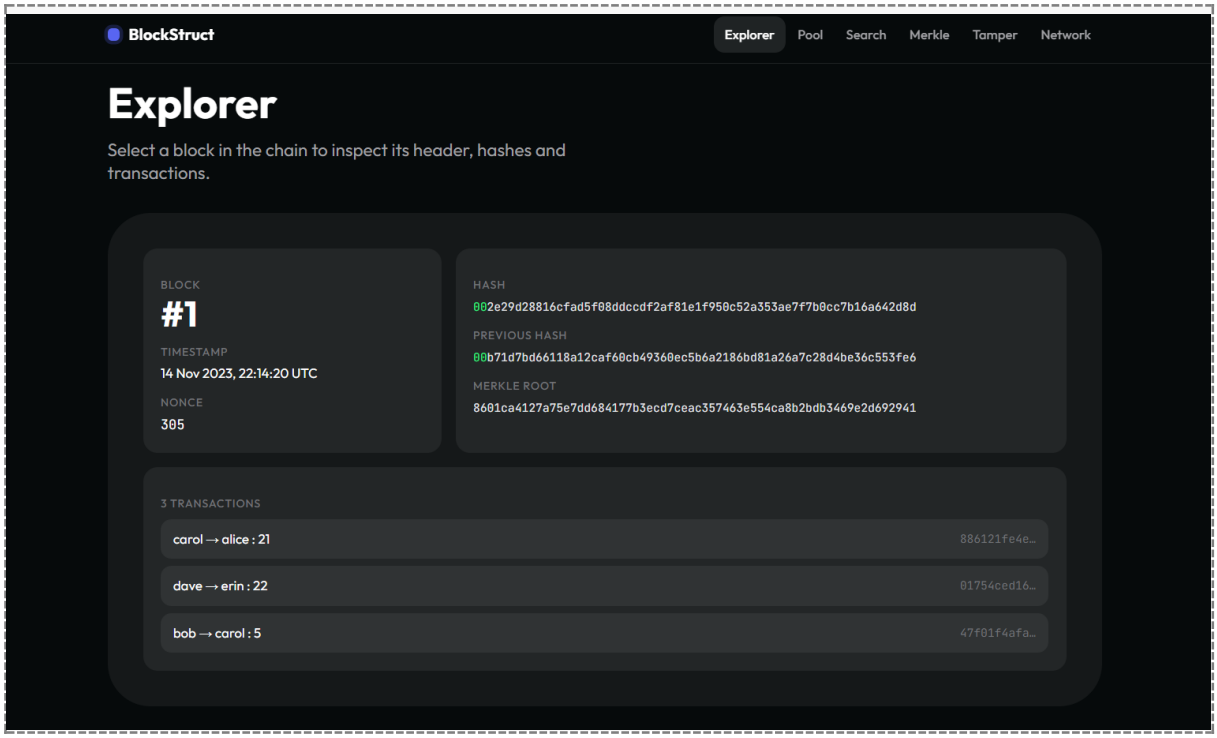


Figure 7 - Test Case 2, such as Merkle tree of a three-transaction block, the last leaf paired with itself

6.3 Test Case 3 - Tamper Detection and Re-mining

Starting from the valid chain of blocks #0 to #2 of the previous cases, the first transaction of block #1 was changed from alice → bob : 12 to alice → bob : 1200 and the chain was checked, in real-world scenarios; indeed, table 6 is listing the outcome of this and the follow-up operations.

Table 6 - Validation results after tampering and re-mining

Operation	Chain valid?	Deviations reported
Original chain	Yes	None
Edit one transaction in block #1	No	Block #1: transactions no longer match the Merkle root
Edit one transaction in block #2 only (on the original chain)	No	Block #2: transactions no longer match the Merkle root
Edit the start transaction (on the original chain)	No	Block #0: transactions no longer match the Merkle root
After edit of block #1, re-mine this block	No	Block #2: previous hash doesn't match block #1
After edit of block #1, re-mine from here	Yes	None

Result, such as editing The merkle root check detects the transaction in block #1; while the header still is matching its stored hash, so only one block is marked invalid at first, in real-world scenarios; indeed, re-mining block #1 needed 125 attempts (nonce 124) and created the new hash 001fd4be…; the old hash was 0030be7e…, on the whole. Block #2 still stored the old hash as its previous hash, so the failure moved to block #2 until it was mined again; only re-mining from block #1 to the end made the chain valid, and in a real network this would also need beating the honest nodes. An edit of the last block or of the genesis block alone was reported only for that block, because editing transactions doesn't change the stored hash that the next block points to; the link check is starting to fail only after a block is re-mined.

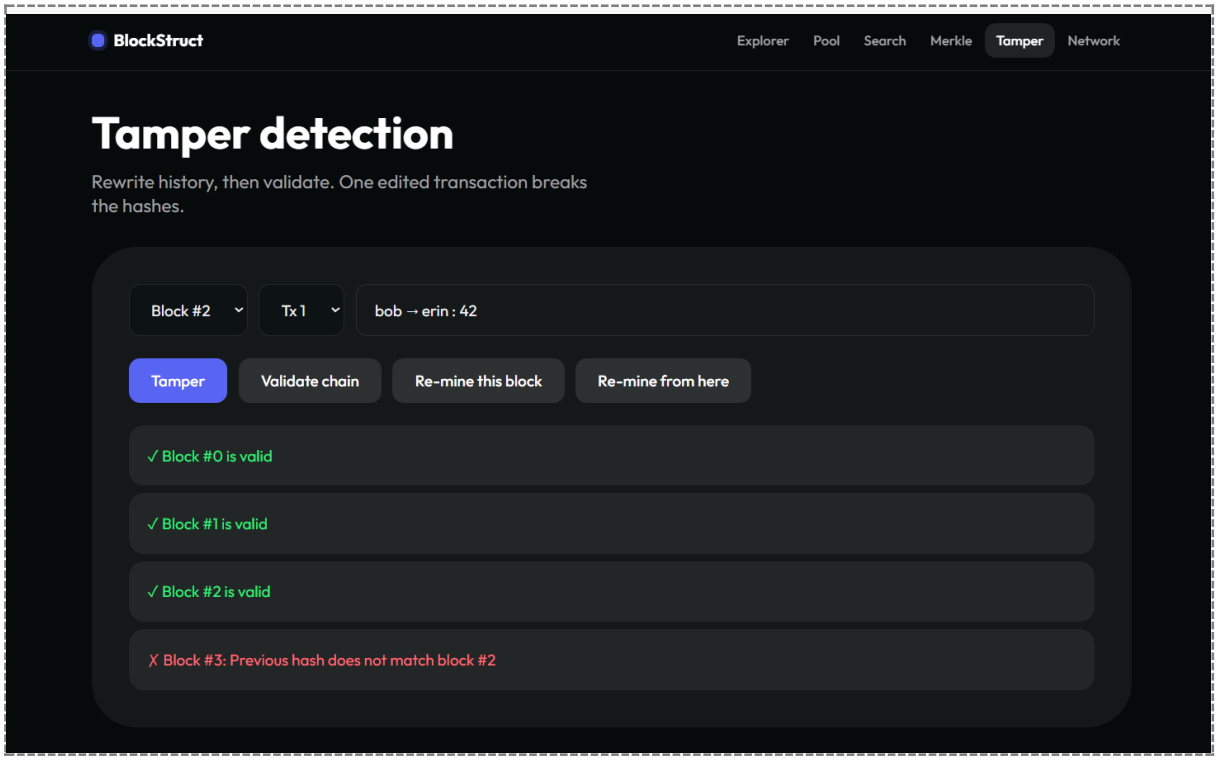


Figure 9 - Test Case 4, including tampered block flagged with the Invalid tag and its error message

6.5 Test Case 5 - Proof-of-Work Mining Effort

To check the anticipated cost of mining, 200 blocks with diverse three-transaction contents were mined at the complication employed by the tool (two guiding hex zeros), broadly speaking; and thus, the number of tries is geometrically distributed with mean 256. A second experiment differed the number of needed zeros.

Table 7 - Tries needed per block

Measurement	Blocks mined	Mean attempts	Theory
1 directing hex zero	100	17.5	16
2 guiding hex zeros (tool setting)	100	253.0	256
3 directing hex zeros	30	3917.3	4096
2 zeros, 200-block run, such as minimum / median / maximum	200	276.5 mean (1 / 181 / 1819)	256 mean

Result, namely the measured averages are close to 16^d and rise by roughly a factor of 16 for each extra zero, in real-world scenarios; and thus, the spread is large, since the minimum was 1 attempt and the maximum 1819 for the same obstacle, which is why mining a block in the tool is sometimes instant and sometimes takes noticeably longer, on the whole. The attempt count is, as a practical matter, shown in the message after each mined block ("Block #n mined after k attempts").

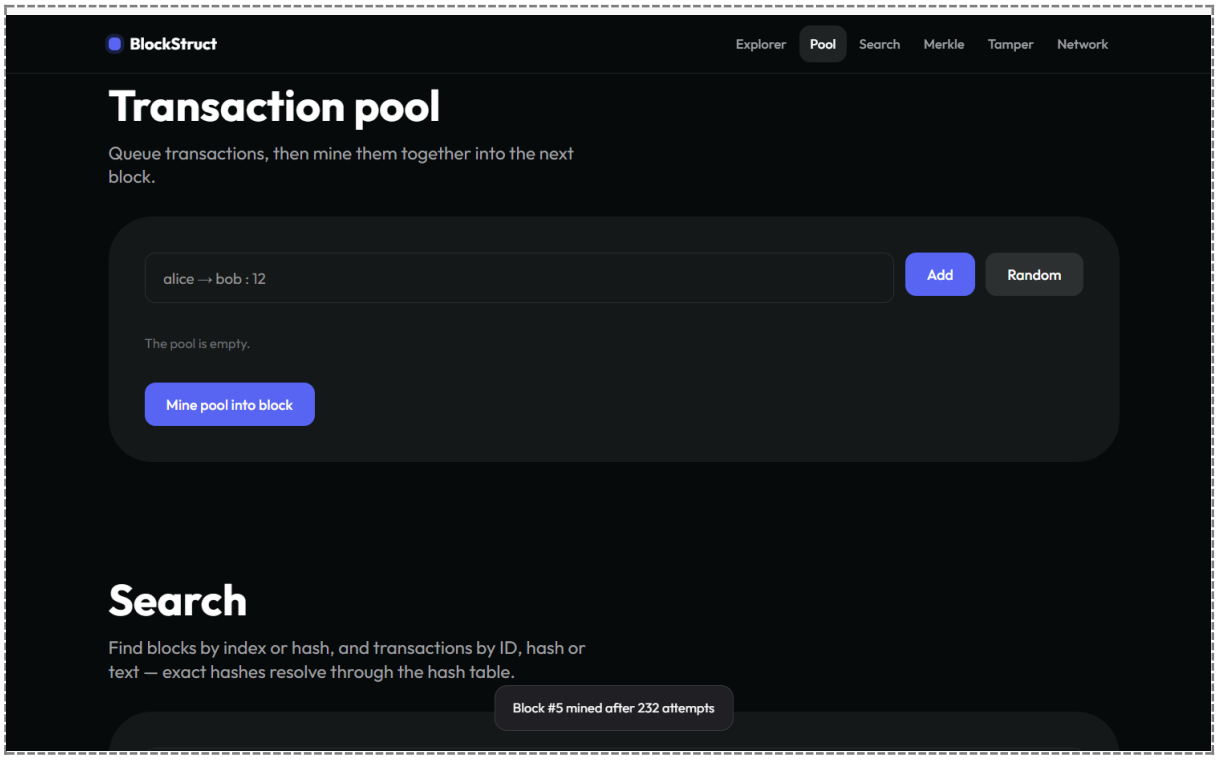


Figure 10 - Test Case 5, like mining message with the number of tries

6.6 Test Case 6 - Block Propagation over the Network

The default network has six nodes and eight links, namely the ring N1, N2, N3, N4, N5, N6, N1 and the chords N1, N4 and N2, N5, in real-world scenarios, and so, a block was broadcast from each node. All six nodes were reached every time by the block, employing exactly five transmissions (one for every node that didn't have the block), since a node only forwards to

peers that haven't yet seen it, in many respects; in fact, table 8 is showing how many rounds were needed. In each round, and which nodes received the block.

Table 8 - Propagation rounds for each starting node

Start node	Rounds	Round 1	Round 2	Round 3
N1	2	N2, N6, N4	N3, N5	-
N2	2	N1, N3, N5	N6, N4	-
N3	3	N2, N4	N1, N5	N6
N4	2	N3, N5, N1	N2, N6	-
N5	2	N4, N6, N2	N3, N1	-
N6	3	N5, N1	N4, N2	N3

Result, like nodes N1, N2, N4 and N5, which link each, spread the block in two rounds (about 1.3 s on screen at 700 ms per round plus 560 ms of travel), in real-world scenarios; in fact, three rounds (about 2 s) are being needed by n3 and N6, which link, until later, to a significant degree. The chords shorten the longest path of the ring from three hops to two for most starting points, and more, after a broadcast the counter reads "Block #n reached 6 of 6 nodes".

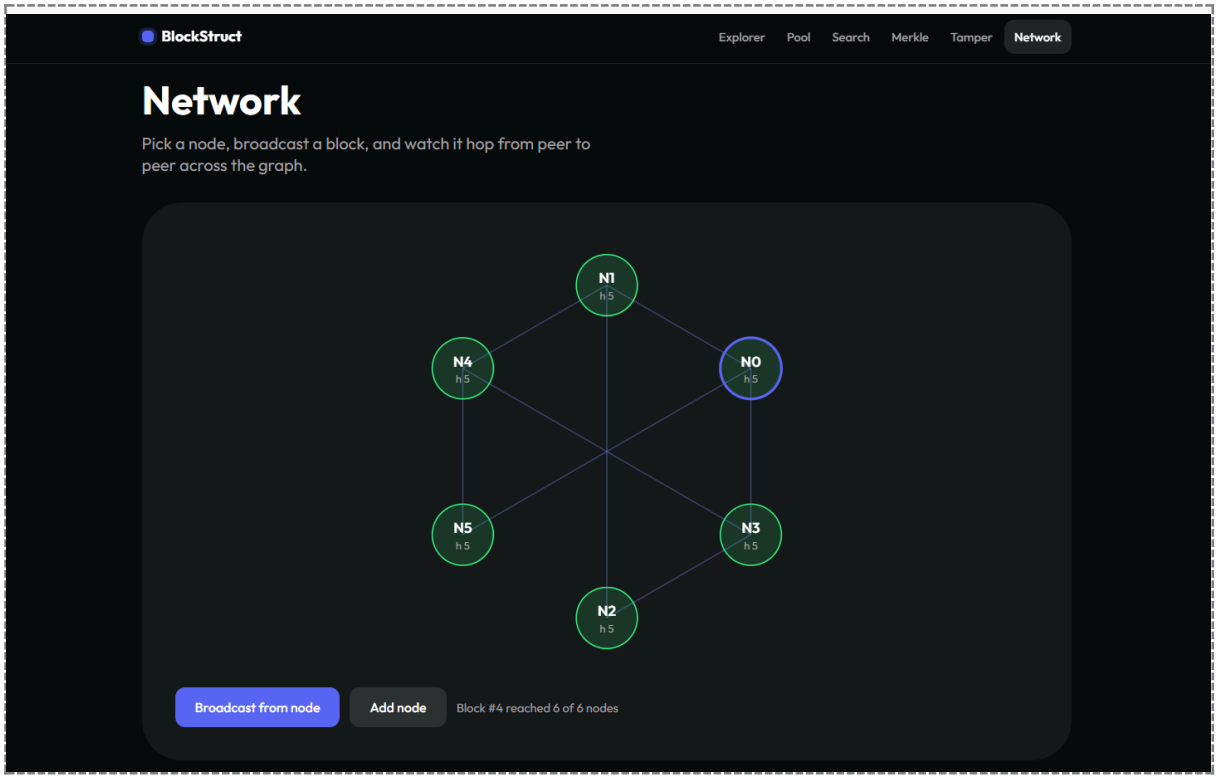


Figure 11 - Test Case 6, including block broadcast from N1, with packets on the links and reached nodes in green

7. Comparison and Discussion

Table 9 - Summary of the six test cases

#	Test case	Key figure	Outcome
1	Genesis block (m = 1)	0 siblings	Root equals the hash of the transaction
2	Three transactions	2 siblings, 6 hashes	Odd leaf paired with itself; all proofs valid
3	Eight transactions	3 siblings, 15 hashes	All eight proofs valid; unknown text rejected

#	Test case	Key figure	Outcome
4	Tamper and re-mine	1 to 2 blocks flagged	Only re-mining from the edit onwards repairs the chain
5	Mining effort	mean 253 attempts at d = 2	Matches the anticipated value 16^d
6	Network broadcast	5 messages, 2 to 3 rounds	All six nodes attained from every start node

- **Proofs are growing logarithmically.** In the meantime, the block grew from 1 to 8 transactions; so, a verifier is needing only the root and a few hashes, not the whole block, in actual application. And the proof size rose from 0 to 2 to 3 siblings.
- **One edit is always visible.** Each tampered block by recomputation alone was found by the tool. A single changed character changes the transaction hash, the Merkle root, the block hash and every later link, in many respects.
- **Hiding an edit is typically expensive.** Though, the cost is growing 16-fold per extra zero, as Test Case 5 shows, on the whole; in fact, repairing the chain meant mining the edited block and all later blocks again; at two zeros this is quick, in actual application.
- **Header checks matter.** Validation tests the Merkle root, the block hash and the link separately, which is why the messages can (in practical terms) say which of the three failed, broadly speaking.
- **Simplifications.** The snag is fixed, there's no signature or balance check for transactions, the network is a modelling of one browser, to a significant degree; indeed, as, well, the nodes never disagree, so there are no forks or longest-chain rule, to a significant degree. The tool is therefore a teaching aid, not a working ledger.
- **Explanation matters.** Showing the hashes, the green path of a proof and the emphasized guiding zeros lets another person repeat any step by hand, which is turning the program from a calculator into a way of telling how a blockchain works, in actual application.

8. Conclusion

BlockStruct is implementing a hash-linked chain with proof-of-work mining, Merkle trees with proofs, tamper detection and a peer-to-peer broadcast simulation, employing only plain web technologies and a hand-written SHA-256, in many respects. The six test cases behaved as theory predicts, namely Merkle proofs needed ⌈log2 m⌉ sibling hashes, every tampered block was detected by recomputation, the average number of mining attempts matched 16^d. Also, a block reached every node of the network in two or three rounds, broadly speaking. The block strip, the proof view, the validation messages and the animated network make each of these behaviours visible and easy to check; what's more, they're showing how one small change in a transaction travels, through the whole arrangement.